Geometric Filtering of LLM-Generated Samples for Few-Shot Text Classification

UAI | FACULTAD DE INGENIERÍA Y CIENCIAS

Benjamín Schindler **Gonzalo A. Ruz**

Universidad Adolfo Ibáñez, Santiago, Chile
Millennium Nucleus for Social Data Science (SODAS)

benjamin.schindler@alumnos.uai.cl
gonzalo.ruz@uai.cl

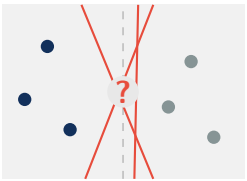
IEEE LACCI 2026

Universidad del Pacífico · Lima, Peru · November 2026

The problem: learning from very few examples

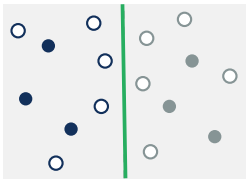
Many real tasks have only **10–50 labeled examples per class**: labeling is slow and costly.

1. Few examples



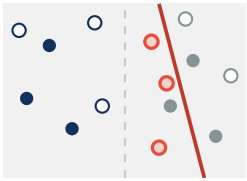
Many boundaries fit the data.
Which one is right?

2. Create more from them



Reliable new examples
pin the boundary down.

3. But only if reliable



Wrong examples
teach the wrong boundary.

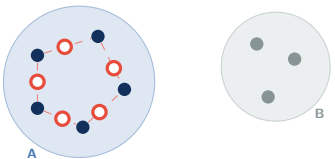
real A · real B · new A · new B · new "A" that is really B · – – true boundary

We need more training data, but only data we can trust.

Two ways to create more training data

SMOTE

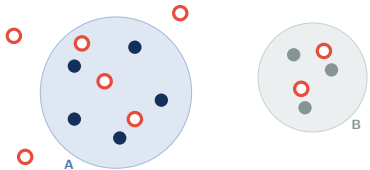
interpolates between real examples



✓ stays in the class ✗ vectors, not text

LLM generation

writes new examples from a prompt

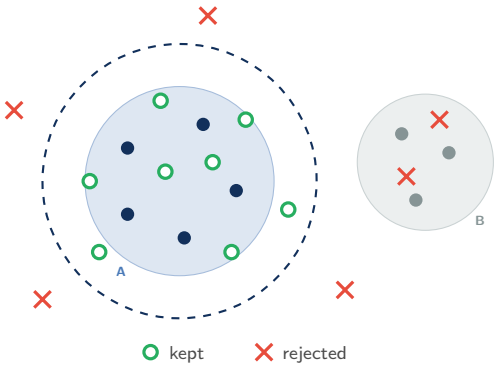


✓ real, fluent text ✗ lands anywhere

real, class A · real, class B · new sample for class A

Can we get LLM text *and* SMOTE's geometric control?

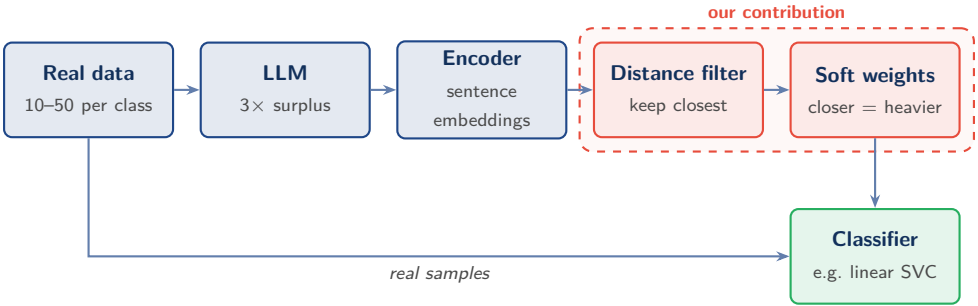
Our idea: keep only the LLM samples that land near their class



1. **Generate** 3× more samples than needed with an LLM.
2. **Embed** them with a sentence encoder.
3. **Keep** the ones closest to the real examples of their class.
4. **Weight** them: closer counts more.

Question: does this beat SMOTE reliably?

The filter only sees embeddings, so any LLM works



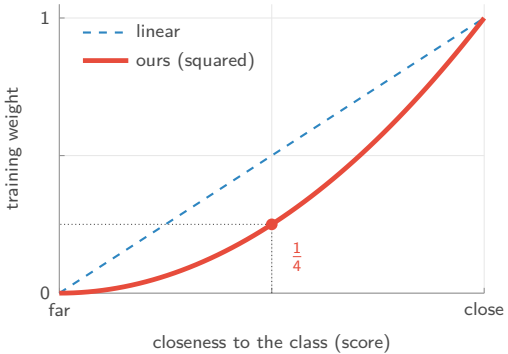
No fine-tuning: just prompting, embedding and filtering.

Closer samples get a larger training weight

Score = how close a sample is to the real examples of its class.

Weight = score^2

A half-way sample counts only **one quarter**.



We tested it broadly



News | Spam | Hate speech | Emotions | Intents | NER | 2–150 classes

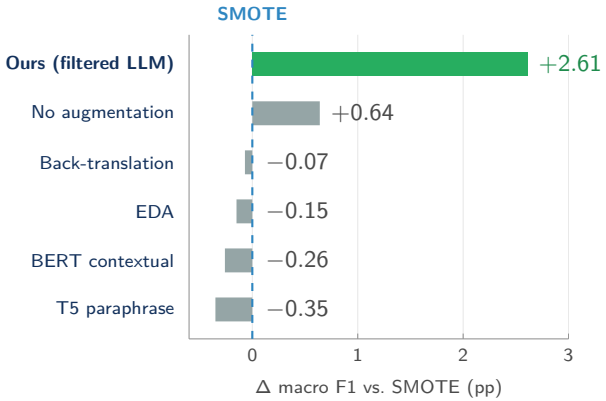
10, 25 or 50 examples per class ▪ macro F1 ▪ paired *t*-tests

Everything is compared against SMOTE.

Filtered LLM data beats SMOTE by +2.6 points

+2.61 pp
macro F1 vs. SMOTE

wins in **89%** of configurations
$p < 0.0001$, large effect

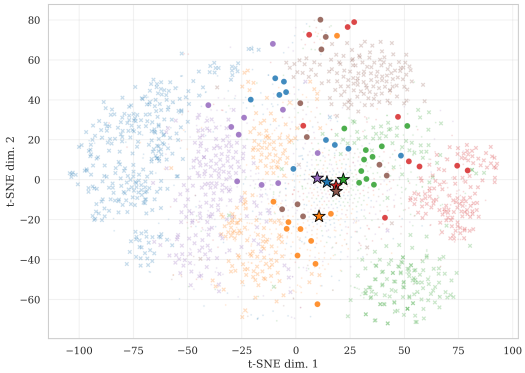


Selected baselines; all methods with 95% CIs in the backup.

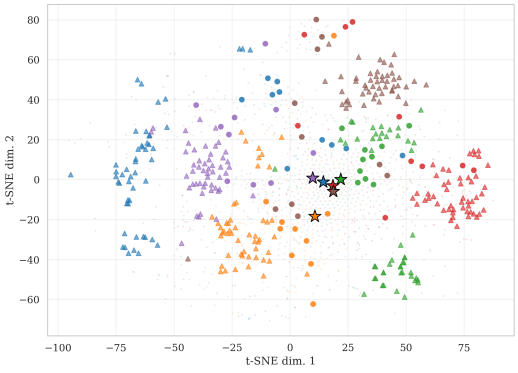
Newer augmentation methods do *not* beat SMOTE. Filtered LLM data does.

The filter removes samples that land in the wrong place

Before: all LLM samples



After: kept samples

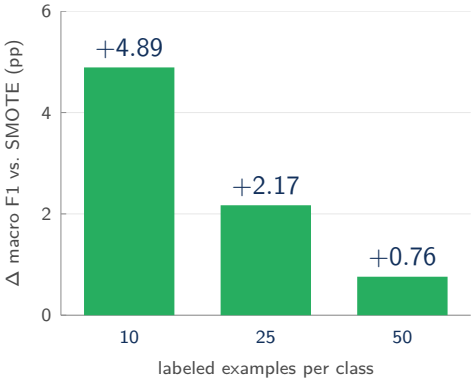


anger fear joy love sadness surprise

emotion dataset, 10 examples per class

Each class becomes a tighter, cleaner cluster.

The fewer labels you have, the more it helps



With **10 examples** per class:

macro F1 **67%** → **72%**

Linear classifiers gain the most (linear SVC: +3.0 pp).

Surprise: the simplest filter works best

Extra metrics are redundant

On normalized embeddings, cosine gives the **same ranking** as distance.

Density needs more data

LOF needs 20 neighbors; with 10 examples a class has only **9**.

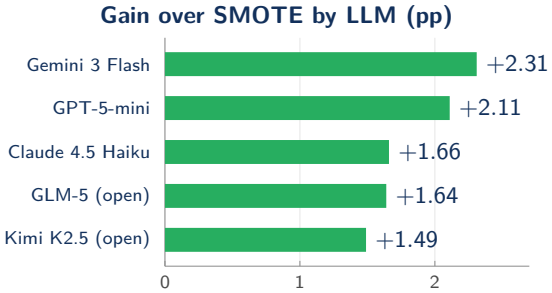
Too strict kills diversity

Combining filters keeps only near-copies, sometimes **nothing at all**.

Distance alone beats LOF, cosine and combined filters.

Generate a surplus, filter lightly. More filtering is not better.

It works with any LLM, and beyond classification



All 5 significant; no LLM is significantly better.

Named entity recognition

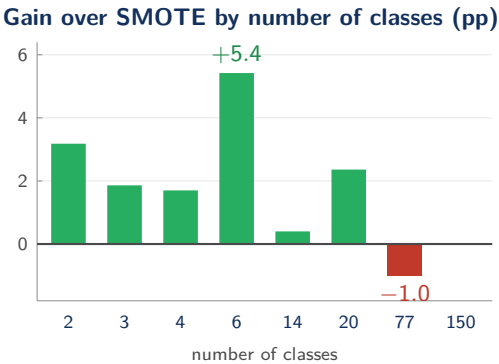
same filter, no changes

+9.26 pp

entity F1 vs. no augmentation

wins in 9 of 9 settings

Best for few labels and few classes



Helps most

- 10–25 examples per class
- up to ∼20 classes
- the hardest classes (up to +10 pp)

Limits

- 77+ classes: no gain
- 50 examples per class: small gain
- tested in English only

Take-home messages

1. **Filtering LLM samples by distance works:** +2.6 pp over SMOTE.
2. **Keep it simple:** distance alone beats complex filters.
3. **Plug and play:** any LLM, and it transfers to NER.

Thank you! Questions?

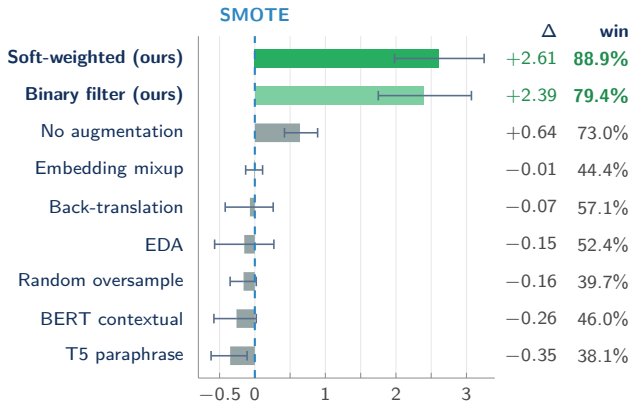
benjamin.schindler@alumnos.uai.cl · gonzalo.ruz@uai.cl

Backup: detailed results

Main result: +2.61 pp over SMOTE

+2.61 pp
macro F1 vs. SMOTE
(soft-weighted)

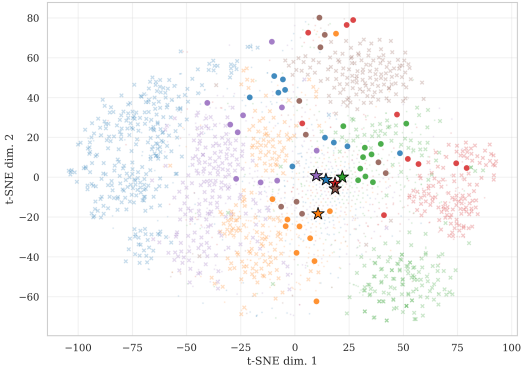
- $p < 0.0001$ [Bonferroni]
- Cohen's $d = 0.95$ [large]
- Win rate = 88.9%
- 95% CI [+1.98, +3.25]



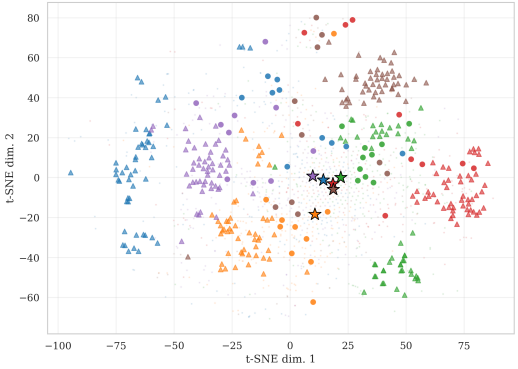
Modern baselines (T5, BERT, mixup) do *not* beat SMOTE; filtered LLM data does.

What the filter does: emotion dataset, 10-shot

Before: all LLM candidates (×)



After: kept by cascade L1 (▲)



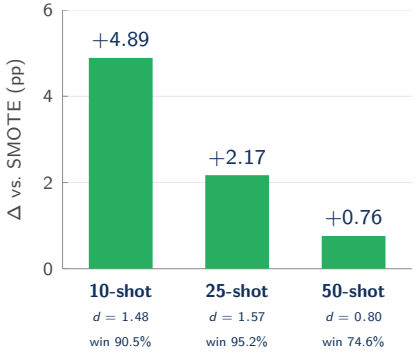
anger fear joy love sadness surprise

● real × candidate ▲ kept ★ centroid

Cross-class and peripheral candidates are discarded; class regions get denser.

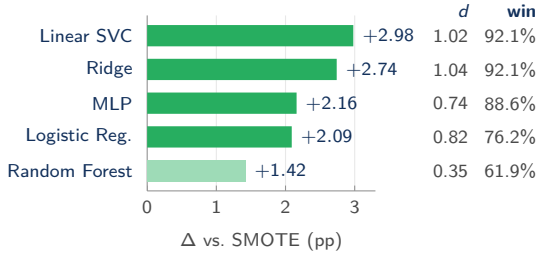
Largest gains where data is scarcest

By examples per class



10-shot macro F1: **67.23%** → **72.12%**

By classifier



Linear models gain most: filtered samples densify class regions near hyperplane boundaries. Axis-parallel tree splits benefit less.

All gains significant; strongest at 10-shot and for linear classifiers.

Counter-intuitive finding: simple beats complex

1. Redundant metrics

For L2-normalized vectors:

$$\|u - v\|_2^2 = 2\,(1 - \cos(u, v))$$

Cosine adds **no new information**: same ranking.

2. LOF breaks at few-shot

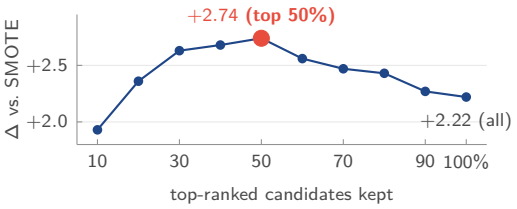
LOF uses $k = 20$ neighbors, but at 10-shot a class has only **9 real neighbors**.

Density estimates degenerate; distance stays stable.

3. Over-filtering

Intersecting criteria keeps only **near-duplicates**.

Combined filter: 0% acceptance in 6 of 9 NER configs; the only filter not significant.



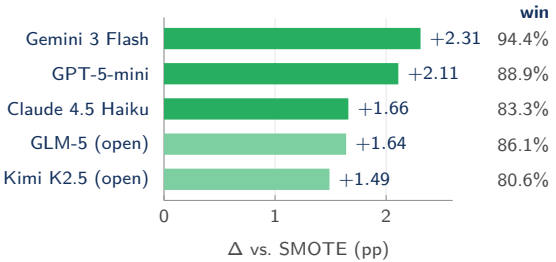
Curriculum check: adding candidates best-first, performance **peaks at the top 50%** and then declines.

The lower half of the ranking adds noise, not signal.

Generate a surplus, filter lightly by distance. More filtering is not better.

Robust across LLMs, and it transfers to NER

By generator LLM [all significant]



Friedman $\chi^2 = 8.15$, $p = 0.086$: no significant LLM effect.

Kendall $W = 0.59$: models agree on which configs benefit.

NER, same filter, **no modification**

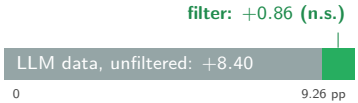
MultiNERD, WikiANN, Few-NERD; class = dominant entity.

+9.26 pp

entity-level F1 vs. no augmentation

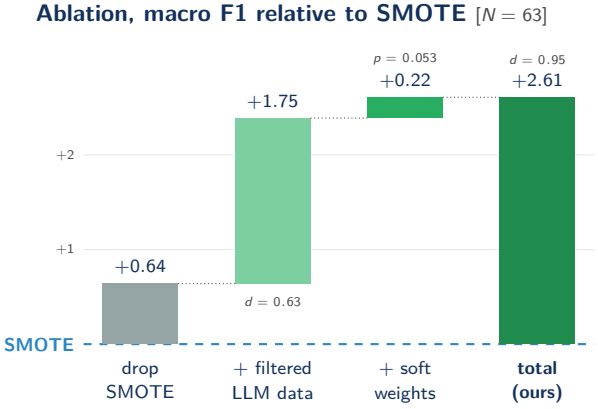
$p_{\text{Bonf}} = 0.002$, $d = 2.05$, **100% win rate** (9/9)

Where the +9.26 pp come from



The filter is decoupled from the LLM and from the task.

Backup: where does the gain come from?



Most of the contribution comes from **geometric selection itself** (+1.75 pp over no augmentation, $p < 0.0001$); soft weighting adds a small, marginally significant refinement (+0.22 pp).

Binary filter vs. SMOTE: +2.39 pp, $d = 0.82$.
No augmentation already beats SMOTE by +0.64 pp.

Caching caveat: cached LLM responses give zero cross-seed variance for deterministic classifiers (80% of configs). Restricted to stochastic classifiers the effect remains: +1.79 pp, $p = 0.0015$, $d = 0.52$.

Backup: NER results (entity-level F1)

Corpus	n	Baseline	No filter	Cascade L1	LOF
MultiNERD	10	.247	.393	.407	**.426**
	25	.315	.419	**.438**	.431
	50	.415	.486	.455	**.495**
WikiANN	10	.162	.268	**.303**	.302
	25	.253	.272	**.299**	.276
	50	.244	**.295**	.287	.293
Few-NERD	10	.103	**.234**	.220	.217
	25	.161	.240	**.251**	.235
	50	.200	.248	**.272**	.261
Average		.233	.317	.326	.326
Δ vs. baseline		—	+8.40	+9.26	+9.28

Cascade L1: $p_{\text{Bonf}} = 0.002$ (7 filters), $d = 2.05$, 100% win rate. Combined filter: 0% acceptance in 6/9 configs, $p = 0.29$.

Backup: protocol details

Generation

- Gemini 3 Flash (primary); GPT-5-mini, Claude 4.5 Haiku, Kimi K2.5, GLM-5.
- Real examples as in-context demonstrations; 3 candidates per desired sample.
- Responses cached (MD5) for reproducibility.

Embeddings

- all-mpnet-base-v2, 768-d, L2-normalized.
- Ablation with bge-large, e5-large, bge-small: 90.5% cross-model agreement.

Protocols

- Main: 10 methods, 3 linear classifiers, 3 seeds (1,890 runs).
- Extended: 7 methods, 5 classifiers, 5 seeds (3,675 runs); MLP and RF rows.
- Seeds: 42, 123, 456, 789, 1011.

Classifiers

- Logistic regression, linear SVC, Ridge, random forest (100 trees), MLP (100 hidden units).

Teacher-model filtering

- Geometric $+2.78$ vs. teacher $+2.43$ pp (binary variants); paired $p = 0.0001$.